

Resumen teórico – Práctico 03

Autoreferencia, Teorema de la Recursión y Oráculos

Computabilidad y Complejidad (Teoría de la Computación II)
FCEFQyN – Universidad Nacional de Río Cuarto
Primer Cuatrimestre de 2026

Resumen teórico-práctico que acompaña al Práctico 03. Fuentes principales: Capítulo 6 del libro de M. Sipser, *Introduction to the Theory of Computation* ([chapter-06.pdf](#)), y láminas de la cátedra ([autoref.pdf](#)).

Índice

1. Motivación: la paradoja de la autoreproducción	2
2. La función q y la máquina SELF	2
2.1. El lema constructivo	2
2.2. La máquina SELF	2
2.3. Analogía textual	2
3. El Teorema de la Recursión	3
4. Aplicaciones clásicas del Teorema de la Recursión	3
4.1. Indecibilidad de A_{TM} (nuevo enfoque)	3
4.2. Las MT minimales no son reconocibles	4
4.3. Teorema del punto fijo	4
5. Introspección en lenguajes de programación	5
5.1. El módulo <code>inspect</code> de Python	5
5.2. Quines: el patron $A + B$	5
5.3. Creación dinámica de clases	5
6. Máquinas con oráculo y reducibilidad de Turing	5
6.1. Definiciones	5
6.2. Propiedades básicas	6
6.3. Ejemplo clásico: $E_{TM} \leq_T A_{TM}$	6
6.4. Límite del poder de los oráculos: argumento de cardinalidad	6
7. Conexión con el práctico	7
Síntesis final	7

1. Motivación: la paradoja de la autoreproducción

Sipser abre el Capítulo 6 con una vieja paradoja: si toda máquina A que produce otra máquina B es necesariamente *más compleja* que B , entonces ninguna máquina podría construirse a sí misma. Como nosotros mismos somos máquinas (en algún sentido biológico) y nos reproducimos, la paradoja debe estar mal planteada.

El **Teorema de la Recursión** resuelve la paradoja mostrando que las máquinas *sí* pueden autoreproducirse, e incluso pueden obtener su propia descripción y operar con ella. Esa capacidad es la base de muchas herramientas: virus informáticos, quines, introspección en lenguajes modernos, y demostraciones limpias de indecibilidad.

2. La función q y la máquina SELF

2.1. El lema constructivo

Lema 2.1 (Sipser 6.1). *Existe una función computable*

$$q : \Sigma^* \longrightarrow \Sigma^*$$

tal que, para toda $w \in \Sigma^$, $q(w) = \langle P_w \rangle$, donde P_w es una Máquina de Turing que, en cualquier entrada, borra la cinta, escribe w y se detiene.*

Intuición. Dada una cadena w , construir “a mano” una MT que la imprima es trivial: cableamos w en una tabla de transiciones que la escribe letra por letra. Lo no obvio del lema es que esa construcción es *computable* en w : hay una MT Q que, recibiendo w , produce la descripción $\langle P_w \rangle$.

2.2. La máquina SELF

Teorema 2.2 (Sipser, páginas 246–248). *Existe una Máquina de Turing SELF que, en cualquier entrada, imprime su propia descripción $\langle \text{SELF} \rangle$ en la cinta y se detiene.*

Construcción. SELF se descompone en dos partes A y B ejecutadas en secuencia:

- **Parte A:** es la máquina $P_{\langle B \rangle}$ del Lema 2.1. Al ejecutarse, deja $\langle B \rangle$ en la cinta.
- **Parte B:** con $\langle B \rangle$ en la cinta:
 1. computa $q(\langle B \rangle) = \langle A \rangle$;
 2. combina $\langle A \rangle$ y $\langle B \rangle$ en $\langle AB \rangle = \langle \text{SELF} \rangle$;
 3. imprime $\langle \text{SELF} \rangle$ y se detiene.

Observar que no hay circularidad: $\langle A \rangle$ no aparece como dato cableado en B ; se *reconstruye en tiempo de ejecución* aplicando q a la cadena $\langle B \rangle$ que dejó A . □

2.3. Analogía textual

Sipser ilustra la técnica con una oración autoreproductiva:

*Copiar dos veces el siguiente párrafo, una con doble comillas:
“Copiar dos veces el siguiente párrafo, una con doble comillas.”*

La parte B es la oración sin comillas; la parte A es la misma oración entre comillas. Al “ejecutar” las instrucciones se obtiene el texto original. La línea con comillas es exactamente $P_{\langle B \rangle}$ (un dato literal); la línea sin comillas es el “programa” que la consume.

3. El Teorema de la Recursión

Teorema 3.1 (Sipser 6.3, recursión). *Sea T una Máquina de Turing que computa una función $t : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$. Entonces existe una MT R que computa una función $r : \Sigma^* \rightarrow \Sigma^*$ tal que, para toda $w \in \Sigma^*$,*

$$r(w) = t(\langle R \rangle, w).$$

Intuición. T es “lo que se quiere hacer” suponiendo que ya se tiene $\langle R \rangle$ a mano; el teorema garantiza que existe una MT R que se comporta como T con su propia descripción automáticamente inyectada en la primera componente. En otras palabras: una MT puede *obtener su propia descripción* y computar con ella.

Esquema. Se construye R en tres partes A, B, T :

- $A = P_{\langle BT \rangle}$: deja $\langle BT \rangle$ en la cinta sin tocar la entrada w .
- B : aplica q a $\langle BT \rangle$, obtiene $\langle A \rangle$, combina con BT para obtener $\langle ABT \rangle = \langle R \rangle$, escribe $\langle R, w \rangle$ y pasa el control a T .
- T termina la computación como en el enunciado.

R resulta así una MT bien definida con la propiedad pedida. □

Diagrama mental

$$\begin{array}{ll} \langle M \rangle, w & \rightarrow T(\langle M \rangle, w) \\ w & \rightarrow R \text{ usa su propia descripción para computar } T(\langle R \rangle, w) \end{array}$$

Cómo usar el teorema

A la hora de diseñar una MT, se puede simplemente incluir en su pseudocódigo la frase “*obtener la propia descripción $\langle M \rangle$ por el Teorema de la Recursión*”. El Teorema garantiza que tal máquina existe.

4. Aplicaciones clásicas del Teorema de la Recursión

4.1. Indecibilidad de A_{TM} (nuevo enfoque)

Una de las consecuencias más elegantes del Teorema de la Recursión es una prueba muy corta de la indecibilidad de

$$A_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ acepta } w \}.$$

Teorema 4.1 (Sipser 6.5). A_{TM} es indecidible.

Demostración. Supongamos, para llegar a una contradicción, que existe una MT H que decide A_{TM} . Definimos:

$B =$ “Con entrada w :

1. obtener su propia descripción $\langle B \rangle$ (Teorema de la Recursión);
2. simular H con entrada $\langle B, w \rangle$;
3. hacer lo contrario de lo que H dice: aceptar si H rechaza, y rechazar si H acepta.”

Para cualquier w , B acepta w si y solo si H rechaza $\langle B, w \rangle$, es decir, si y solo si B no acepta w . Contradicción. Luego H no puede existir. $\square$

Observacion. La prueba evita la diagonalización de Cantor (Teorema 4.11) y reemplaza el truco de “aplicarse al propio código” por la herramienta general del Teorema de la Recursión. El resultado es el mismo, pero el argumento se vuelve casi mecánico.

4.2. Las MT minimales no son reconocibles

Definición 4.2 (Sipser 6.6). Si M es una MT, la *longitud* de $\langle M \rangle$ es la cantidad de símbolos de la cadena que la describe. Decimos que M es *minimal* si no existe una MT equivalente con descripción más corta. Definimos

$$MIN_{TM} = \{ \langle M \rangle \mid M \text{ es minimal} \}.$$

Teorema 4.3 (Sipser 6.7). MIN_{TM} no es Turing-reconocible.

Demostración. Supongamos que un enumerador E enumera MIN_{TM} . Construimos:

$C =$ “**Con entrada w :**

1. obtener su propia descripción $\langle C \rangle$ (Recursión);
2. ejecutar E hasta que aparezca una MT D con $|\langle D \rangle| > |\langle C \rangle|$;
3. simular D sobre w .”

Como MIN_{TM} es infinito, eventualmente E imprime una MT D con $|\langle D \rangle| > |\langle C \rangle|$. Por construcción, C es *equivalente* a D (simula a D paso a paso). Pero $|\langle C \rangle| < |\langle D \rangle|$, luego D no es minimal. Contradicción con $\langle D \rangle \in MIN_{TM}$. $\square$

4.3. Teorema del punto fijo

Teorema 4.4 (Sipser 6.8, punto fijo). Sea $t : \Sigma^* \rightarrow \Sigma^*$ computable. Existe una MT F tal que $t(\langle F \rangle)$ describe una MT equivalente a F .

Esquema. Definimos F :

1. Obtener $\langle F \rangle$ (Recursión).
2. Computar $t(\langle F \rangle)$, que es la descripción de cierta MT G .
3. Simular G sobre la entrada.

Entonces F y G tienen comportamientos idénticos, y $t(\langle F \rangle) = \langle G \rangle$ describe una MT equivalente a F . $\square$

El teorema dice que toda transformación computable de descripciones de MT tiene un *punto fijo semántico*: alguna MT cuyo comportamiento queda inalterado por la transformación. Es el análogo del Teorema de Rice en su forma más primitiva.

5. Introspección en lenguajes de programación

El Teorema de la Recursión garantiza la posibilidad de *obtener la propia descripción*. En lenguajes de programación reales esta capacidad se llama **introspección** (o *reflection*).

5.1. El módulo `inspect` de Python

Python expone esta capacidad mediante:

- `inspect.getsource(obj)`: devuelve el código fuente de `obj` (función, clase, etc.). Es la versión real de “obtener $\langle M \rangle$ ”.
- `inspect.getmembers(obj)`: lista todos los miembros de un objeto: atributos, métodos, dunders.
- `inspect.signature(función)`: devuelve la firma (parámetros y anotaciones).
- `inspect.isfunction`, `inspect.isclass`, etc.: predicados para clasificar miembros.

Ejemplo mínimo:

```
import inspect
def saludar(nombre: str) -> str:
    return f"Hola, {nombre}"
print(inspect.getsource(saludar))
print(inspect.signature(saludar))
```

5.2. Quines: el patron $A + B$

Un *quine* es un programa que imprime su propio código fuente. La forma canónica usa el patron $A + B$ de SELF. En Python:

```
B = 'B = {!r}\nprint(B.format(B))\n'
print(B.format(B))
```

Aquí `repr` cumple el rol de la función q del Lema 2.1: toma una cadena y produce el literal Python que la representa.

5.3. Creación dinámica de clases

Python permite construir clases en tiempo de ejecución con la built-in `type(nombre, bases, atributos)`. Es exactamente el constructor que invoca internamente la sentencia `class`:

```
Punto = type("Punto", (object,), {
    "__init__": lambda self, x, y: setattr(self, "x", x) or setattr(self, "y", y),
    "norma": lambda self: (self.x ** 2 + self.y ** 2) ** 0.5,
})
```

Esto es la versión programable de la construcción del Lema 2.1: a partir de una *descripción* (nombre, bases, diccionario) se produce un nuevo objeto ejecutable.

6. Máquinas con oráculo y reducibilidad de Turing

6.1. Definiciones

Definición 6.1 (Sipser 6.18, oráculo). Un *oráculo* para el lenguaje B es un dispositivo externo capaz de reportar, en un único paso, si una cadena w pertenece o no a B . Una *MT con oráculo*

M^B es una MT modificada con la posibilidad de hacer consultas a tal oráculo. Formalmente se modela con un par de estados especiales q_{ask} , q_{yes} , q_{no} y una cinta adicional para escribir la consulta.

Definición 6.2 (Sipser 6.20, Turing reducibility). A es *Turing reducible* a B , notado $A \leq_T B$, si A es decidable relativo a B ; es decir, existe una MT con oráculo M^B que decide A .

6.2. Propiedades básicas

Teorema 6.3 (Sipser 6.21). Si $A \leq_T B$ y B es decidable, entonces A es decidable.

Demostración. Sea M^B un decisor relativo a B y D_B un decisor (real) de B . Una MT estandar M' simula a M^B reemplazando cada consulta al oráculo por una simulación completa de D_B . M' decide A sin oráculos. $\square$

Observación. $\leq_T$ **generaliza a** $\leq_m$. Si $A \leq_m B$ con reducción f , construimos una MT con oráculo B que dada x computa $f(x)$ y consulta “ $f(x) \in B$?”. La respuesta del oráculo es la respuesta para “ $x \in A$?”. Luego toda mapping reduction es una Turing reduction.

6.3. Ejemplo clásico: $E_{TM} \leq_T A_{TM}$

Ejemplo 6.4 (Sipser 6.19). $E_{TM} = \{\langle M \rangle \mid \mathcal{L}(M) = \emptyset\}$ es decidable relativo a A_{TM} .

$T^{A_{TM}} =$ “Con entrada $\langle M \rangle$:

1. construir la MT $N =$ “En cualquier entrada: ejecutar M en paralelo sobre todas las cadenas de Σ^* ; aceptar si alguna es aceptada por M ”;
2. consultar al oráculo si $\langle N, 0 \rangle \in A_{TM}$;
3. si la respuesta es NO, aceptar; si es SÍ, rechazar.”

Si $\mathcal{L}(M) \neq \emptyset$, N acepta toda entrada y la consulta da SÍ; T rechaza. Si $\mathcal{L}(M) = \emptyset$, N nunca acepta y la consulta da NO; T acepta. Por lo tanto $T^{A_{TM}}$ decide E_{TM} .

6.4. Límite del poder de los oráculos: argumento de cardinalidad

Aún teniendo un oráculo poderoso, existen lenguajes inalcanzables. La clave es contar.

Teorema 6.5 (Sipser, Problema 6.19). Existen lenguajes $L \subseteq \Sigma^*$ que no son reconocidos por ninguna MT con oráculo A_{TM} .

Demostración. La familia $\mathcal{M}_{A_{TM}}$ de MT con oráculo A_{TM} es contable (cada una se describe por una cadena finita). Por lo tanto el conjunto de lenguajes reconocidos por elementos de $\mathcal{M}_{A_{TM}}$ es contable. En cambio, $\mathcal{P}(\Sigma^*)$ es incontable (Corolario 4.18). Por comparación de cardinalidades existe algún $L \in \mathcal{P}(\Sigma^*)$ fuera de esa familia. $\square$

Observación. El argumento es *robusto bajo cambios de oráculo*: para cualquier $B \subseteq \Sigma^*$ existen lenguajes no reconocibles por MT con oráculo B . Esto da una idea de la *jerarquía aritmética* y de los grados de Turing.

7. Conexión con el práctico

Mapa ejercicio $\leftrightarrow$ teoría

Ej.	Tema	Herramienta teórica principal
2	quine en Python	Lema 2.1 y SELF
3	dos MT que se imprimen mutuamente	Lema 2.1 + Teorema 3.1
4	lenguajes no reconocibles con $M^{A_{TM}}$	Teorema 6.5
5	introspección de una clase Python	inspect como análogo de “obtener $\langle M \rangle$ ”
6	creación dinámica de una clase	type como análogo de q

Resumen ejecutivo de la técnica de autoreferencia

1. Si necesitamos una MT que use $\langle M \rangle$, escribimos su pseudocódigo usando libremente la frase “obtener $\langle M \rangle$ ” y aplicamos el Teorema 3.1 para garantizar su existencia.
2. Si necesitamos una MT que imprima cierto texto fijo w , usamos P_w del Lema 2.1.
3. Si necesitamos una MT que produzca otra MT a partir de su propia descripción, combinamos ambos: SELF + transformación computable τ .

Pseudocódigo unificado

M = "En entrada x:

1. (Si se necesita) obtener $\langle M \rangle$ por el Teorema de la Recursion.
2. Computar $f(\langle M \rangle, x)$ (o cualquier función computable usando $\langle M \rangle$).
3. Imprimir, simular, aceptar/rechazar, etc."

Síntesis final

El Capítulo 6 de Sipser y las láminas `autoref.pdf` introducen tres ideas que se entrelazan:

- **Autoreferencia:** una MT puede obtener su propia descripción (Teorema de la Recursión). De aquí se derivan, mecánicamente, SELF, los quines, la no reconocibilidad de MIN_{TM} y una nueva prueba de indecidibilidad de A_{TM} .
- **Oráculos y $\leq_T$:** extender una MT con un oráculo es la forma más general de la noción intuitiva de reducción. Sin embargo, agregar un oráculo no permite escapar de los límites cardinales: existen lenguajes incontables y oráculos numerables.
- **Introspección:** en lenguajes Turing-completos reales (Python es el ejemplo canónico), las facilidades de **inspect** y **type** hacen tangibles las construcciones de la teoría.

En conjunto, este práctico convierte un tema aparentemente filosófico (“¿puede un programa hablar de sí mismo?”) en una herramienta operativa de la teoría de la computación y de la programación real.